PROPOSTA DI PROGETTO DI RETE

Compagnia Theta

Architettura di rete, sicurezza perimetrale, preventivo di spesa e strumenti di analisi

PROPOSTA N.	001/2026
DATA DI EMISSIONE	26/10/2026
CLIENTE	Compagnia Theta
APPALTATORE	Zero-Day Smokey S.r.l. Via del Rampino 4, Italia

NOTA CONTRATTUALE

La presente Proposta di Progetto di Rete (di seguito “PPR”) è emessa in esecuzione dell’accordo per la prestazione di servizi di consulenza stipulato tra Compagnia Theta (di seguito “Cliente”) e Zero-Day Smokey S.r.l. (di seguito “Appaltatore”). Con la sottoscrizione da parte di entrambe le Parti, la PPR costituisce contratto vincolante tra le stesse, con efficacia a decorrere dal 26/10/2026. Eventuali modifiche al testo devono essere comunicate all’altra Parte, prima della sottoscrizione, a mezzo PEC o raccomandata con avviso di ricevimento.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

INDICE

1. Architettura di rete 3
1.1 Architettura campus ad alta affidabilità (Collapsed Core) 3
1.2 Configurazione degli apparati 5
1.3 Piano di indirizzamento e subnetting 7
1.4 Sicurezza perimetrale 7
1.5 Collegamento inter-sede: studio di fattibilità 9
1.6 Verifica e collaudo dell'infrastruttura 11

2. Regole del firewall 12
2.1 Creazione delle regole firewall 12
2.2 Estensione alla Sede 2 17

3. ZMAP: il toolkit di rete 18
3.1 Introduzione 18
3.2 Librerie utilizzate 18
3.3 Struttura generale del programma 18
3.4 Descrizione delle funzioni 19
3.5 Blocco principale e menu 21
3.6 Uso etico e legale 22

4. Raccomandazioni di sicurezza 23
4.1 Infrastruttura di rete 23
4.2 Firewall 23

5. Strumento di cattura dei socket di rete (Sniffer) 24
5.1 Funzionamento in sintesi 24
5.2 Import dei moduli 24
5.3 Funzioni di supporto 25
5.4 Creazione del socket raw 25
5.5 Ciclo di cattura e header Ethernet 26
5.6 Decodifica dell'header IP 27
5.7 Decodifica di TCP, UDP e ICMP 28
5.8 Esecuzione e note d'uso 28

6. Preventivo di spesa 30

7. Enigma 31
7.1 Analisi della cartella file della Compagnia Theta 31

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

1. ARCHITETTURA DI RETE

Il presente capitolo descrive l'architettura di rete progettata per la Compagnia Theta, articolata su due edifici situati nella zona PIP (Piccoli Insediamenti Produttivi) del Comune di Cavenago di Brianza e distanti circa 250 metri l'uno dall'altro. Sono illustrati il modello di distribuzione adottato, la configurazione dei principali apparati, il piano di indirizzamento, la sicurezza perimetrale e lo studio di fattibilità del collegamento tra le due sedi. Le regole del firewall sono trattate nel capitolo 2, mentre il preventivo di spesa è riportato nel capitolo 6.

1.1 Architettura campus ad alta affidabilità (Collapsed Core)

La figura seguente riporta la topologia complessiva dell'infrastruttura: a sinistra la Sede 1 (nuova acquisizione, sei piani), al centro il nucleo della rete con il blocco firewall – router – Internet e la zona demilitarizzata (DMZ), a destra la Sede 2 (vecchio stabile restaurato, tre piani).

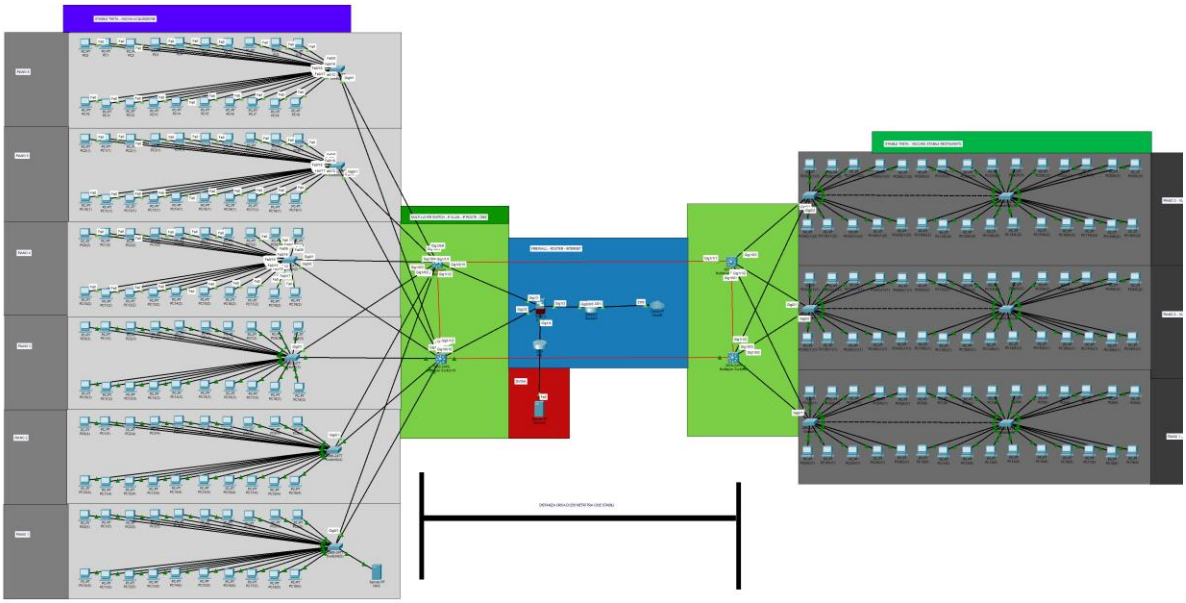


Figura 1 – *Schema topologico dell'infrastruttura di rete delle due sedi.*

1.1.1 Panoramica dell'architettura

Il progetto prevede la realizzazione di un'infrastruttura di rete aziendale distribuita su due sedi, secondo il paradigma del Campus Model Compresso (Collapsed Core), con elevata ridondanza. I servizi DHCP sono integrati negli switch core e consentono la gestione dinamica degli indirizzi IP.

- **Sede 1 (edificio principale, sei piani)**: ospita venti computer per piano, per un totale di 120 postazioni, ciascuno collegato a uno switch di piano dedicato. Gli switch di piano sono attestati ai due Multilayer Switch del nucleo di rete (Core 1 e Core 2), che fungono

da router centrale. Nella Sede 1 sono inoltre collocati il firewall perimetrale, il router edge verso Internet, la DMZ con il server DVWA e il NAS, connesso allo switch del piano terra, in prossimità del nucleo, affinché i dati siano accessibili a tutti i computer aziendali.

- **Sede 2 (edificio secondario, tre piani)**: ospita trenta computer per piano, per un totale di 90 postazioni, serviti da switch di piano e da una coppia di Multilayer Switch (Core 3 e Core 4) distribuiti su tre piani con le relative VLAN di piano. Il traffico della Sede 2 è instradato verso la Sede 1 mediante il collegamento in fibra ottica ridondato descritto al paragrafo 1.5.
- **DMZ unica**: l'intera infrastruttura utilizza una sola DMZ, ubicata nell'edificio della Sede 1. La Sede 2 non dispone di una DMZ propria e raggiunge i servizi pubblicati attraverso il perimetro della Sede 1.
- **Servizi DHCP integrati**: i Multilayer Switch di ciascuna sede operano come server DHCP locali per le rispettive VLAN. I pool di indirizzi sono definiti escludendo i primi indirizzi, riservati alle interfacce SVI e agli indirizzi virtuali HSRP.

1.1.2 Motivazione del modello Collapsed Core (architettura a due livelli)

Per il campus distribuito su due sedi è stata preferita la topologia Collapsed Core al tradizionale modello a tre livelli (Core, Distribution, Access). Tale scelta progettuale comporta i seguenti vantaggi operativi e prestazionali:

- **Riduzione della latenza**: l'unificazione dei livelli Core e Distribution in un unico layer multilivello riduce il numero di hop di routing per il traffico inter-VLAN e inter-sede, migliorando i tempi di risposta delle applicazioni.
- **Ottimizzazione dei costi e della gestione**: riduce in misura significativa i costi di acquisizione dell'hardware, il consumo energetico e gli oneri di manutenzione, semplificando la topologia senza compromettere le funzionalità.
- **Eliminazione dei Single Point of Failure (SPOF)**: la ridondanza attiva-passiva a livello di gateway (HSRP), unita alle doppie dorsali in fibra ottica tra le sedi, garantisce la continuità del servizio anche in caso di guasto di un intero switch core o di un cavo di dorsale.

1.1.3 Sistemi e tecnologie utilizzati

- **Multilayer Switch**: costituiscono il centro nevralgico della rete, combinando la commutazione Layer 2 ad alta velocità con l'instradamento Layer 3 inter-VLAN eseguito direttamente in hardware. L'integrazione dei server DHCP sui medesimi apparati riduce al minimo la latenza nell'assegnazione degli indirizzi IP.
- **HSRP (Hot Standby Router Protocol)**: fornisce ridondanza trasparente per i gateway predefiniti delle VLAN. In caso di malfunzionamento dello switch primario, lo switch secondario subentra in tempi molto ridotti, senza interruzioni percepibili dagli utenti.

ZERO-DAY SMOKEY NETWORK DESIGN & ENGINEERING

- **Rapid-PVST+ (Per-VLAN Spanning Tree Plus)**: previene la formazione di loop Layer 2 sui collegamenti trunk ridondati e assicura tempi di convergenza della topologia molto rapidi in caso di guasto di un collegamento.
- **Firewall perimetrale**: presidia la sicurezza perimetrale gestendo tre zone di fiducia distinte: Inside (livello 100, rete interna protetta), DMZ (livello 50, servizi pubblici o isolati) e Outside (livello 0, rete esterna/Internet). Applica l'ispezione stateful del traffico e il Port Address Translation (PAT) dinamico.
- **Router edge**: termina la connessione verso il provider Internet (WAN/ISP) e gestisce l'instradamento verso l'esterno. Integra una funzione IPS interna (cfr. paragrafo 1.4.6).
- **IPS dedicato**: apparato di prevenzione delle intrusioni collocato tra il firewall e il server DVWA, a protezione della DMZ (cfr. paragrafo 1.4.6).

1.2 Configurazione degli apparati

1.2.1 Modello Collapsed Core e ruoli degli apparati

Il design di rete adotta un'architettura Collapsed Core a due livelli, al fine di ottimizzare i percorsi di instradamento, ridurre le latenze intra-sito e consolidare la gestione dell'infrastruttura.

1. **Architettura distribuita**: l'infrastruttura si articola su due sedi mediante quattro Multilayer Switch. Sede 1: Core Switch 1 e Core Switch 2; Sede 2: Core Switch 3 e Core Switch 4.
2. **Resilienza e alta disponibilità (HA)**: l'integrazione funzionale del livello Distribution all'interno del Core consente di rimuovere qualsiasi singolo punto di guasto (Single Point of Failure – SPOF), garantendo continuità operativa e bilanciamento del carico tra i nodi ridondati.

Tabella 1 – Ruoli degli apparati core.

Sede	Dispositivo	Ruolo primario	Tipologia di connessione
Sede 1	Core 1	Core/Distribution primario	Interconnessione L3 con il firewall e uplink verso Sede 2
Sede 1	Core 2	Core/Distribution secondario (HSRP Standby)	Ridondanza locale e cross-link
Sede 2	Core 3	Core/Distribution primario della sede remota	Dorsale inter-sede punto-punto
Sede 2	Core 4	Core/Distribution secondario della sede remota	Ridondanza locale inter-sede

SMOKEY
NETWORK DESIGN & ENGINEERING

1.2.2 Conversione delle interfacce a Livello 3 (no switchport)

Al fine di ottimizzare le prestazioni ed eliminare l'overhead introdotto dai protocolli di switching (STP) sui segmenti critici di dorsale e perimetrali, le interfacce selezionate sono configurate in modalità routed.

- **Ambito di applicazione**: porte fisiche di transito verso il firewall perimetrale e collegamenti dedicati punto-punto (dorsali inter-sede).
- **Meccanismo operativo**: il comando `no switchport` disabilita l'incapsulamento Layer 2 (802.1Q) e lo switching di trama, trasformando la porta in un'interfaccia instradabile a Layer 3, alla quale è assegnato direttamente un indirizzo IP dedicato (`ip address <IP> <Subnet>`).

Listato 1 – Configurazione di un'interfaccia routed.

```
interface GigabitEthernet0/1
 description UPLINK-ROUTED-TO-FIREWALL
 no switchport
 ip address 10.0.0.1 255.255.255.252
 no shutdown
```

1.2.3 Ridondanza first-hop e Spanning Tree

La stabilità dell'accesso e la continuità di servizio per i nodi terminali si fondano sulla combinazione di protocolli FHRP e di meccanismi anti-loop avanzati.

Hot Standby Router Protocol (HSRP)

Il protocollo è configurato a livello delle Switch Virtual Interface (SVI, ad esempio `interface Vlan10`) per offrire un gateway predefinito condiviso e trasparente (Virtual IP):

1. **Switch primario**: priorità elevata a 110 (`standby 10 priority 110`), con abilitazione del ripristino automatico del ruolo (`standby 10 preempt`).
2. **Switch secondario**: priorità standard pari a 100, in modalità standby attivo.

Rapid Per-VLAN Spanning Tree Plus (Rapid-PVST+)

Il protocollo è attivo a livello globale sui collegamenti L2 di distribuzione e accesso, per prevenire la formazione di loop di commutazione e garantire una convergenza deterministica in caso di modifiche topologiche o di guasto di un collegamento.

1.2.4 Servizi di assegnazione dinamica (DHCP)

I Multilayer Switch Core svolgono la funzione di server DHCP per i segmenti utente delle rispettive sedi.

1. **Esclusione e riserva degli indirizzi**: gli indirizzi critici (VIP HSRP, indirizzi fisici di router e switch, stampanti di rete e server fissi) sono isolati preventivamente tramite la direttiva `ip dhcp excluded-address <IP_Inizio> <IP_Fine>`.

ZERO-DAY SMOKEY NETWORK DESIGN & ENGINEERING

2. **Configurazione degli scope**: ogni VLAN utente dispone di un pool dedicato, che definisce lo spazio di indirizzamento e indica come default router il gateway virtuale HSRP.

Listato 2 – Esempio di esclusione indirizzi e pool DHCP per la VLAN 10.

```
ip dhcp excluded-address 192.168.10.1 192.168.10.10
!
ip dhcp pool VLAN10-CLIENTS
 network 192.168.10.0 255.255.255.0
 default-router 192.168.10.1
 dns-server 1.1.1.1 8.8.8.8
```

1.3 Piano di indirizzamento e subnetting

Il dimensionamento delle subnet è stato eseguito sul numero di postazioni di ciascun piano, aggiungendo gli indirizzi necessari alle interfacce SVI dei due core (due indirizzi), all'indirizzo virtuale HSRP (uno) e un margine per stampanti e dispositivi di rete.

Tabella 2 – Dimensionamento delle subnet di piano.

Sede	Piani	PC per piano	Indirizzi minimi per piano	Subnet minima	Subnet adottata
Sede 1	6 (VLAN 10–60)	20	20 + 3 + margine	/27 (30 host utili)	/24 – 192.168.N.0 per ciascuna VLAN
Sede 2	3 (VLAN 70–90)	30	30 + 3 + margine	/26 (62 host utili)	/24 – 192.168.N.0 per ciascuna VLAN

Pur essendo teoricamente sufficienti una subnet /27 per i piani della Sede 1 e una /26 per quelli della Sede 2, è stata adottata una subnet /24 per ciascuna VLAN. La scelta è motivata dalla semplicità di gestione (il terzo ottetto coincide con il numero della VLAN), dal margine di crescita e dalla possibilità di riassumere l'intera rete interna nel prefisso 192.168.0.0/16 nelle regole di instradamento, NAT e filtraggio. Le reti di collegamento punto-punto tra gli apparati utilizzano subnet /30 (ad esempio 192.168.210.0/30 e 192.168.211.0/30 per i collegamenti tra i core e il firewall, 192.168.100.0/30 tra firewall e router edge); la DMZ utilizza una subnet /24 dedicata (192.168.150.0/24).

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

1.4 Sicurezza perimetrale

1.4.1 Firewall perimetrale e zone di sicurezza

Il firewall perimetrale è posizionato tra il router interno e la connessione a Internet. La segmentazione è governata da un modello a zone gerarchiche basato sui Security Levels (da 0 a 100).

Tabella 3 – Zone di sicurezza del firewall perimetrale.

Zona	Security Level	Ruolo e descrizione	Policy di transito predefinita
Inside	100	Rete interna fidata, connessa ai Core Switch della Sede 1 (e, tramite essi, alla Sede 2).	Accesso autorizzato verso i livelli inferiori (DMZ e Outside).
DMZ	50	Segmento cuscinetto contenente i servizi esposti (server DVWA).	Accesso consentito verso Outside (0); bloccato verso Inside (100).
Outside	0	Rete esterna non attendibile, attestata sul router edge/ISP.	Tutto il traffico in ingresso è negato (implicit deny).

1.4.2 Sintesi della configurazione del firewall

Il firewall dispone di quattro interfacce: due collegamenti punto-punto verso i Core Switch della Sede 1 (percorso primario e percorso di ridondanza, entrambi in zona Inside), il collegamento verso il router edge (zona Outside) e il collegamento verso la DMZ. Le reti interne (192.168.10.0/24 – 192.168.90.0/24) sono raggiunte tramite rotte statiche con metrica preferenziale sul percorso primario e metrica più elevata sul percorso di ridondanza, mentre la rotta predefinita è diretta al router edge. Il traffico verso l'esterno è tradotto mediante PAT dinamico sull'indirizzo dell'interfaccia Outside.

Dynamic Interface PAT (Auto NAT)

La tecnica consente di mascherare cumulativamente tutte le workstation interne di entrambe le sedi durante le sessioni di navigazione verso la rete esterna, mediante l'indirizzo IP pubblico associato alla porta Outside.

Listato 3 – Configurazione del PAT dinamico ad interfaccia.

```
object network OBJ-RETE-INTERNA
 subnet 192.168.0.0 255.255.0.0
 nat (inside,outside) dynamic interface
```

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

1.4.3 Configurazione del router edge

Il router edge gestisce la connessione verso l'esterno (Cloud/ISP), con rotta di ritorno verso il firewall:

Listato 4 – Configurazione di base del router edge.

```
hostname RTV-EDGE
ip routing

interface GigabitEthernet0/0/0
 description LINK-TO-FIREWALL-OUTSIDE
 ip address 192.168.100.1 255.255.255.252
 no shutdown

interface GigabitEthernet0/0/1
 description LINK-TO-CLOUD-ISP
 ip address dhcp
 no shutdown

ip route 192.168.0.0 255.255.0.0 192.168.100.2
```

1.4.4 DMZ unica

L'infrastruttura utilizza una sola DMZ, ubicata nell'edificio della Sede 1 (sei piani), tra il firewall e la connessione a Internet. Nella DMZ è collocato il web server (DVWA di Metasploitable), raggiungibile dall'esterno in condizioni di sicurezza e isolato dalla rete interna. La Sede 2 non dispone di una DMZ propria: i suoi utenti accedono al web server attraverso il Collapsed Core e il perimetro della Sede 1, e il traffico originato dalla DMZ non può raggiungere le reti interne di nessuna delle due sedi.

1.4.5 Collegamento del NAS

Il NAS è collegato allo switch del piano terra della Sede 1, in prossimità dei Core Switch, e risulta raggiungibile da tutti i computer aziendali: quelli della Sede 1 direttamente, quelli della Sede 2 attraverso il collegamento in fibra ottica tra le sedi.

1.4.6 Sistemi di prevenzione delle intrusioni (IPS)

Per il monitoraggio del traffico e la prevenzione delle intrusioni sono implementati due IPS, con funzioni e posizioni complementari:

1. **IPS integrato nel router edge**: funzione interna al router, che ispeziona il traffico in transito tra Internet e il perimetro aziendale e blocca le comunicazioni corrispondenti a firme di attacco note, prima che raggiungano il firewall.
2. **IPS dedicato tra il firewall e la DVWA**: apparato posizionato in linea sul segmento che collega il firewall al server DVWA, a protezione della DMZ. Ispeziona il traffico diretto al web server (la componente più esposta della rete) e quello da esso originato, rilevando tentativi di sfruttamento delle vulnerabilità applicative e comportamenti anomali di un eventuale server compromesso.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

La combinazione dei due sistemi realizza una difesa a più livelli: il primo IPS filtra il traffico ostile alla frontiera della rete, il secondo presidia la zona a maggiore rischio.

1.5 Collegamento inter-sede: studio di fattibilità

1.5.1 Requisiti

La dirigenza di Theta richiede una connessione tra le due sedi, situate nella zona PIP del Comune di Cavenago di Brianza a circa 250 metri di distanza, che sia quanto più veloce possibile. Lo studio di fattibilità riguarda la scelta della tecnologia, i costi degli apparati, la posa dei mezzi trasmissivi e la gestione dei vincoli burocratici, normativi e dei permessi comunali necessari per attraversare l'area industriale.

1.5.2 Scelta della tecnologia

Sono state valutate le seguenti alternative.

Tabella 4 – Confronto tra le tecnologie di collegamento.

Soluzione	Prestazioni	Vantaggi	Criticità
Fibra ottica multimodale di proprietà (scelta adottata)	Da 1 a 10 Gbit/s su 250 m, con margini di crescita	Massima velocità, bassissima latenza, immunità ai disturbi elettromagnetici, nessun canone ricorrente	Opere di scavo e permessi comunali
Fibra ottica monomodale di proprietà	Oltre 10 Gbit/s, portata di chilometri	Massima scalabilità	Costo di fibra e ottiche superiore, non giustificato da una distanza di 250 m
Ponte radio punto-punto	Da centinaia di Mbit/s a pochi Gbit/s	Nessuno scavo, rapidità di installazione	Sensibilità a interferenze e a condizioni meteo, necessità di visibilità ottica, prestazioni inferiori
Collegamento in affitto da operatore	Dipende dal contratto	Nessuna opera civile a carico del Cliente	Canone ricorrente, dipendenza dal fornitore

È stata scelta la fibra ottica multimodale di proprietà. Le ottiche multimodali a 850 nm garantiscono, su fibra di tipo OM3, portate di 550 m a 1 Gbit/s e di 300 m a 10 Gbit/s, ampiamente superiori ai 250 m da coprire, e consentono di raggiungere la massima velocità oggi sostenibile dagli apparati previsti. Il collegamento è realizzato con almeno due fibre su percorsi attestati a core diversi (rispettivamente Core 1–Core 3 e Core 2–Core 4), in coerenza con il principio di eliminazione dei Single Point of Failure.

ZERO DAY SMOKEY
NETWORK DESIGN & ENGINEERING

1.5.3 Apparati e mezzi trasmissivi

- **Cavo in fibra ottica multimodale** per circa 250 m, con margine per le curve del percorso e per gli attestamenti.
- **Moduli ottici SFP multimodali a 850 nm** con connettore LC duplex, uno per ciascuna porta di dorsale dei Multilayer Switch (due per sede).

I costi puntuali sono riportati nel preventivo (capitolo 6).

1.5.4 Attività tecniche previste

Le attività a carico dell'Appaltatore, comprese nel preventivo, sono le seguenti:

1. **Pratica e progettazione**: redazione del progetto esecutivo del collegamento e della documentazione tecnica per gli enti competenti.
2. **Posa della fibra** all'interno del cavidotto predisposto, attestamento e installazione dei moduli ottici sui Multilayer Switch.
3. **Collaudo con certificazione** del collegamento.

Le opere civili e murarie necessarie alla predisposizione del percorso (scavi, cavidotti, ripristini) non sono comprese nel preventivo e restano a carico del Cliente o di impresa edile da questi incaricata.

1.5.5 Vincoli burocratici, normativi e permessi comunali

L'attraversamento dell'area industriale comporta l'attivazione di una serie di adempimenti, da avviare prima dell'inizio dei lavori, il cui esito condiziona il cronoprogramma:

- **Autorizzazione all'esecuzione dello scavo e all'occupazione del suolo**: richiesta all'Ufficio Tecnico (o allo Sportello Unico per le Attività Produttive) del Comune di Cavenago di Brianza, con progetto del tracciato, sezioni di scavo e piano di ripristino; se la viabilità interna alla zona PIP è di proprietà privata o consortile, è necessario anche il consenso del soggetto proprietario o gestore della strada.
- **Normativa in materia di infrastrutture di comunicazione elettronica**: la realizzazione di infrastrutture di rete è disciplinata dal Codice delle comunicazioni elettroniche e dalle norme che favoriscono la posa con tecniche di scavo a ridotto impatto (minitrincea); le prescrizioni applicabili al caso specifico devono essere verificate con il Comune.
- **Interferenze con i sottoservizi**: coordinamento con gli enti gestori delle reti presenti sul tracciato, sulla base dell'indagine preventiva.
- **Viabilità e sicurezza**: eventuale ordinanza di limitazione o di regolamentazione del traffico durante i lavori, piano di sicurezza e nomina del coordinatore, ove richiesti.
- **Ripristino e garanzie**: l'Amministrazione può richiedere una cauzione o una fideiussione a garanzia del corretto ripristino del suolo; il collaudo finale e la certificazione del collegamento chiudono la pratica.

ZERO DAY SMOKEY
NETWORK DESIGN & ENGINEERING

> **Nota.** I tempi di rilascio dei permessi dipendono dall'istruttoria degli uffici competenti e non sono interamente controllabili dall'Appaltatore; eventuali oneri, diritti di segreteria, cauzioni e prescrizioni aggiuntive sono trattati nel capitolo 6 (variazioni di spesa ed esclusioni).

1.6 Verifica e collaudo dell'infrastruttura

- **Test HSRP**: simulando il guasto del Core 1, il Core 2 subentra mantenendo attivi i gateway virtuali.
- **Test NAT** (`show xlate`): inviando traffico da un qualsiasi PC (Sede 1 o Sede 2) verso l'esterno, il firewall traduce dinamicamente l'indirizzo IP sorgente sull'interfaccia Outside.
- **Test del collegamento inter-sede**: verifica del collegamento in fibra mediante misure di attenuazione e certificazione, e prova di failover disconnettendo una delle due dorsali.
- **Test delle regole del firewall**: descritti nel capitolo 2.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

2. REGOLE DEL FIREWALL

Il presente capitolo documenta la definizione e la verifica delle regole del firewall perimetrale. Poiché le regole non fanno parte della topologia fisica descritta nel capitolo 1, esse sono trattate separatamente. La validazione è stata condotta in un ambiente di laboratorio virtuale basato su pfSense, che riproduce la zona Inside, la DMZ e la zona Outside del progetto.

2.1 Creazione delle regole firewall

Per l'infrastruttura di rete della Compagnia Theta è stato configurato un firewall perimetrale basato su pfSense, posizionato tra la rete interna e Internet. Il firewall ha il compito di controllare tutto il traffico tra le diverse zone della rete e di proteggere i sistemi aziendali da eventuali attacchi provenienti da Internet.

2.1.1 Architettura a tre zone

La rete è suddivisa in tre zone separate, con pfSense in posizione centrale con funzione di gateway e di controllo per ciascuna di esse: la LAN interna (PC aziendali), la LAN2/DMZ (web server) e la WAN (collegamento verso Internet). Ogni macchina virtuale è collegata alla rete interna della propria zona tramite le schede di rete di VirtualBox; la console di pfSense mostra le tre interfacce configurate.

Tabella 5 – Zone di rete e indirizzamento dell'ambiente di prova.

Interfaccia	Rete IP	Indirizzo pfSense	Contenuto
LAN interna (em1)	192.168.50.0/24	192.168.50.1	PC aziendali Kali Linux – 192.168.50.100
LAN2/DMZ (em2)	192.168.2.0/24	192.168.2.1	Web server Metasploitable 2 (DVWA) – 192.168.2.100
WAN (em0)	10.0.2.0 (DHCP, NAT VirtualBox)	10.0.2.15	Collegamento esterno

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING



Figura 2 – *Console di pfSense con le tre interfacce WAN, LAN e LAN2/DMZ e i rispettivi indirizzi IP.*

2.1.2 Motivazione dell'ubicazione del web server in DMZ

Il web server deve essere raggiungibile dall'esterno e, per tale ragione, costituisce la componente più esposta ad attacchi. Collocandolo in una DMZ, in caso di compromissione l'attaccante resta confinato in tale zona, senza poter raggiungere i PC della rete interna aziendale. Come indicato nel capitolo 1, il progetto prevede una sola DMZ, ubicata nell'edificio della Sede 1.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

2.1.3 Regole firewall applicate

Di seguito sono illustrate le impostazioni delle regole firewall applicate.

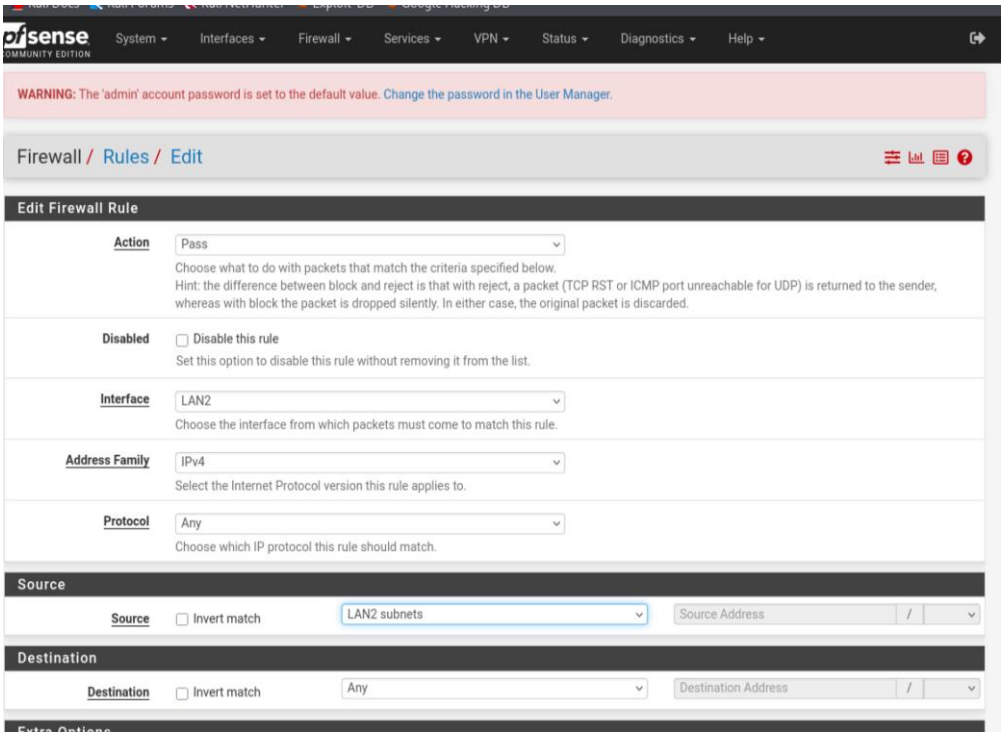


Figura 3a – *Regola Pass che consente il transito del traffico in uscita dalla LAN2/DMZ. (parte superiore)*

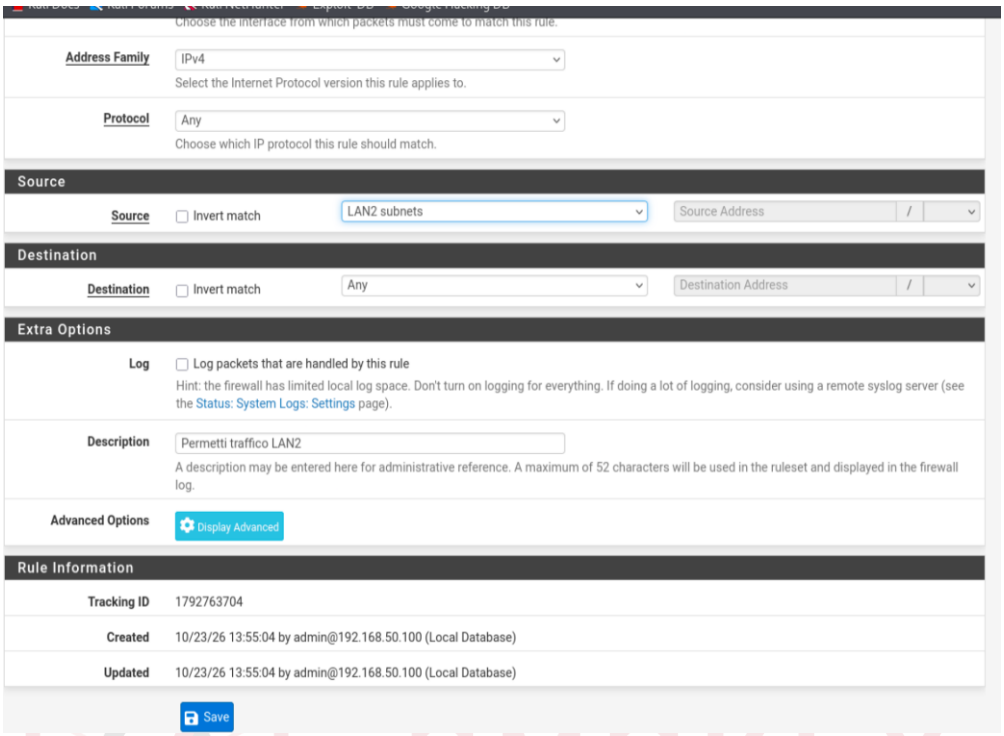


Figura 3b – *Regola Pass che consente il transito del traffico in uscita dalla LAN2/DMZ. (parte inferiore)*

> **Nota.** La schermata è stata acquisita alle 11:29, prima dell'abilitazione dell'opzione "Log packets that are handled by this rule". L'opzione è stata attivata successivamente, come documentato dalla voce di log delle 12:18:53, attribuita alla medesima regola (Tracking ID 1793013870).

È stata creata una regola Pass che consente il transito del traffico con sorgente LAN2 sia verso la LAN interna sia verso la WAN. Per ragioni di sicurezza, tuttavia, al di sopra di tale regola è stata inserita una regola Block che impedisce alla LAN2 di raggiungere la LAN interna. L'ordine delle regole è determinante: pfSense applica la prima regola che corrisponde al pacchetto; il traffico DMZ → LAN viene pertanto bloccato prima di raggiungere la regola Pass. Di seguito sono illustrate le impostazioni della regola Block.

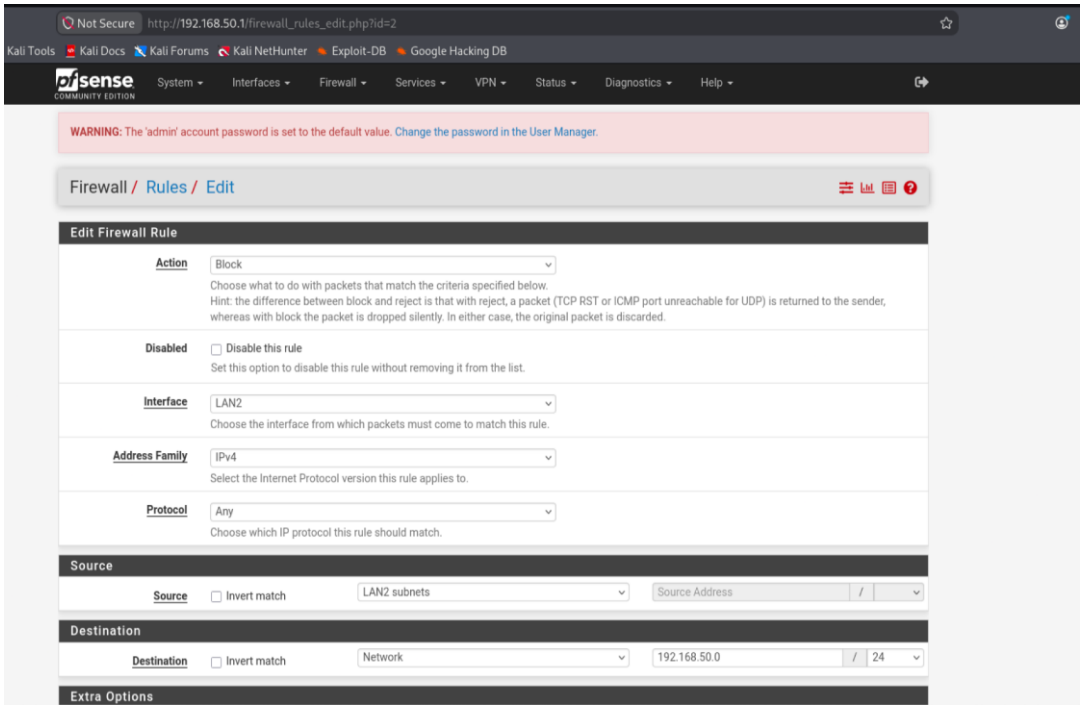


Figura 4a – *Regola Block che impedisce alla LAN2/DMZ di raggiungere la rete interna. (parte superiore)*

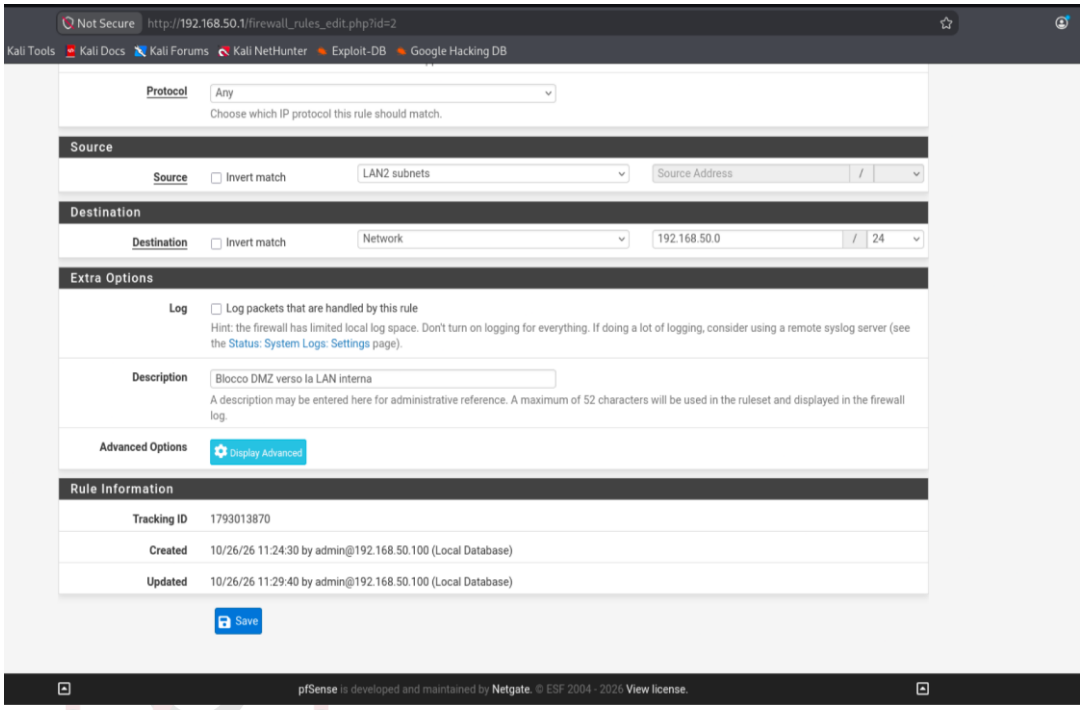


Figura 4b – *Regola Block che impedisce alla LAN2/DMZ di raggiungere la rete interna. (parte inferiore)*

2.1.4 Verifica del funzionamento

Il corretto isolamento è stato verificato mediante i test di seguito descritti.

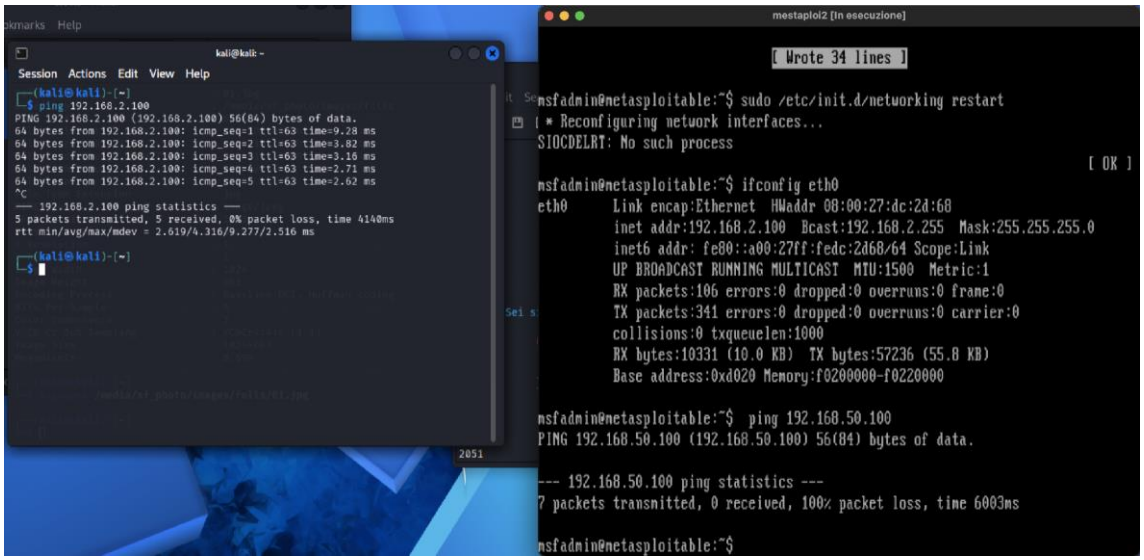


Figura 5 – *Test di connettività. A sinistra: ping da Kali (LAN) verso il web server (192.168.2.100), con risposta corretta. A destra: ping dal web server (DMZ) verso Kali (192.168.50.100), senza risposta (100% packet loss); il server in DMZ non raggiunge pertanto la rete interna.*

Dal browser di Kali è stata aperta correttamente la pagina del web server (http://192.168.2.100), che mostra la pagina iniziale di Metasploitable 2 con il collegamento a DVWA: il web server in DMZ risulta pertanto raggiungibile dalla LAN interna.

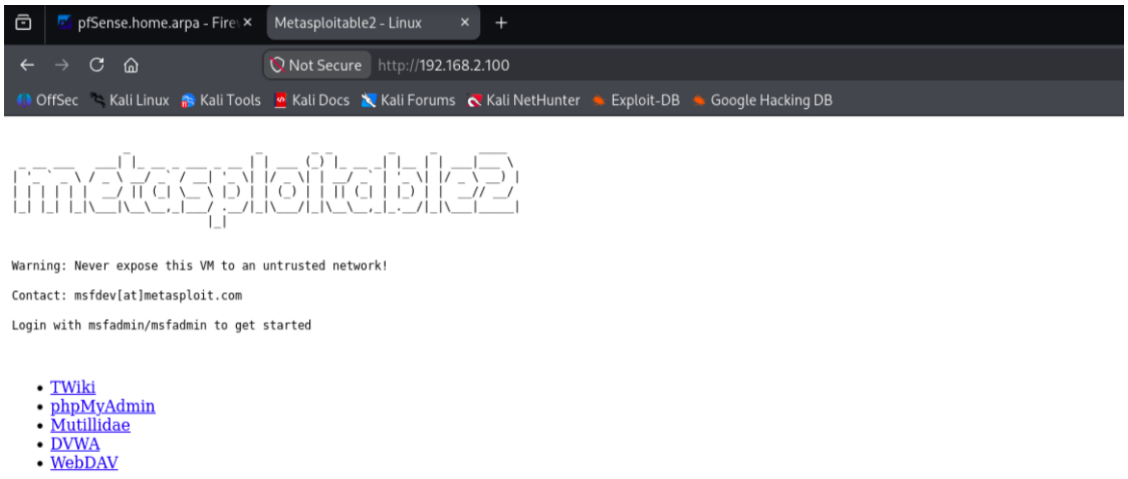


Figura 6 – *Pagina iniziale di Metasploitable 2 raggiunta dalla LAN interna.*

Nei log del firewall (Status / System Logs / Firewall) è presente il pacchetto ICMP inviato dalla macchina in DMZ (Metasploitable 2, 192.168.2.100) verso Kali (192.168.50.100), scartato sull'interfaccia LAN2 dalla regola "Blocco DMZ verso la LAN interna". Il Tracking ID riportato nel log (1793013870) coincide con quello della regola Block illustrata in precedenza: ciò

conferma che il mancato collegamento è imputabile alla regola del firewall e non ad altre cause.



Action	Time	Interface	Rule	Source	Destination	Protocol
✕	Oct 26 12:18:53	LAN2	Blocco DMZ verso la LAN interna (1793013870)	192.168.2.100	192.168.50.100	ICMP

Figura 7 – *Voce di log relativa al pacchetto ICMP scartato dalla regola Block.*

2.2 Estensione alla Sede 2

La Sede 2 (tre piani, 90 computer) adotta i medesimi criteri di sicurezza della Sede 1, senza una DMZ propria e senza un secondo firewall perimetrale: il traffico della Sede 2 verso Internet e verso il web server transita attraverso il Collapsed Core e il perimetro della Sede 1, ed è pertanto soggetto alle stesse regole.

- La regola Block, che impedisce alla DMZ di raggiungere la rete interna, ha come destinazione l'insieme delle subnet interne di entrambe le sedi (192.168.10.0/24 – 192.168.90.0/24), e non la sola LAN della Sede 1.
- Le postazioni della Sede 2 (VLAN 70–90) raggiungono il web server in DMZ con le stesse modalità delle postazioni della Sede 1, come verificato per la LAN interna al paragrafo 2.1.4.
- Il traffico originato dalla DMZ resta bloccato verso qualsiasi rete interna, indipendentemente dalla sede di destinazione.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

3. ZMAP: IL TOOLKIT DI RETE

3.1 Introduzione

Zmap è un programma scritto in Python che consente di eseguire le seguenti operazioni: verifica della raggiungibilità di un host, scansione di una subnet, interrogazione DNS, elenco delle interfacce di rete, scansione delle porte TCP e test del protocollo HTTP.

Lo strumento è stato sviluppato integralmente da zero, senza ricorrere a strumenti esistenti quali nmap.

3.2 Librerie utilizzate

Tabella 6 – Librerie impiegate dal programma.

Libreria	Ruolo nel programma
pyfiglet	Genera il titolo in ASCII art mostrato all'avvio (funzione `figlet_format`).
socket	Componente centrale dello strumento: apre connessioni TCP, risolve nomi di dominio e consente di verificare porte e host.
argparse	Predisposta per un'eventuale gestione degli argomenti da riga di comando.
requests	Invia richieste HTTP (GET, POST, DELETE, HEAD, OPTIONS).
colorama	Consente di colorare l'output del terminale.
subprocess	Esegue comandi del sistema operativo (ad esempio `ip a`).
sys	Fornisce l'accesso a variabili e funzioni per interagire con l'interprete.
time	Introduce brevi pause (sleep).
threading	Consente di eseguire l'animazione di attesa in parallelo alla scansione, tramite un thread e un oggetto Event.

Dopo gli import viene richiamata la funzione `init()` di colorama, che inizializza la gestione dei colori.

3.3 Struttura generale del programma

Il programma è organizzato in modo semplice e modulare:

- una **funzione per ogni funzionalità** (`ping`, `scan`, `dns_lookup`, `list_interfaces`, `port_scan`, `http_test`) più una funzione per il titolo (`print_title`);
- un **blocco principale** (`if __name__ == "__main__":`) che mostra il titolo, presenta un menu numerato in un ciclo `while True` e, in base alla scelta dell'utente, richiama la funzione corrispondente.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

Il ciclo prosegue fino a quando l'utente seleziona l'opzione 7 (Esci), che interrompe il `while` con `break` e chiude il programma.

Tutti i messaggi sono colorati secondo la seguente convenzione: **verde** per le operazioni riuscite, **rosso** per errori o esiti negativi, **blu** per titoli e messaggi informativi, **bianco** per le richieste di input, **ciano** e **giallo** per le intestazioni.

3.4 Descrizione delle funzioni

3.4.1 print_title()

La funzione costruisce e stampa il titolo del programma. Il testo "Zmap by Zero-Day Smokey" viene convertito in ASCII art mediante `figlet_format`.

3.4.2 ping(host)

La funzione verifica se l'host è raggiungibile in due passaggi:

1. **Risoluzione DNS**: con `socket.getaddrinfo(host, None)` si verifica che il nome (o l'indirizzo) fornito sia risolvibile.
2. **Connessione TCP**: si crea un socket IPv4 (`AF_INET`) di tipo stream (`SOCK_STREAM`), si imposta un timeout di 2 secondi e si tenta la connessione alla **porta 80**.

Se la connessione riesce, viene stampato in verde che l'host è raggiungibile e la funzione restituisce `True`. In caso di qualsiasi eccezione (ad esempio nome non risolvibile, timeout, connessione rifiutata) viene stampato in rosso il messaggio di errore e la funzione restituisce `False`. L'uso del blocco `with` garantisce la chiusura automatica del socket al termine della funzione.

3.4.3 scan()

La funzione esegue la scansione di una subnet alla ricerca di host raggiungibili:

1. mostra un messaggio informativo e richiede all'utente di inserire la subnet in notazione CIDR (ad esempio 192.168.1.0/24);
2. percorre gli indirizzi della rete con un ciclo `for` da 1 a 255;
3. per ogni indirizzo tenta una connessione con `socket.create_connection` sulla porta 80, con timeout di 2 secondi;
4. se la connessione riesce, stampa in verde che l'indirizzo è raggiungibile; in caso contrario stampa in rosso che non è raggiungibile, indicando anche l'errore;
5. al termine stampa un messaggio di scansione completata.

3.4.4 dns_lookup(domain)

La funzione interroga il sistema di risoluzione dei nomi per ottenere le informazioni associate a un dominio. Utilizza `socket.gethostbyname_ex(domain)`, che restituisce il nome dell'host, gli eventuali alias e gli indirizzi IP associati. I risultati sono mostrati in blu sotto

l'intestazione "Record DNS", con le singole voci in verde; al termine compare il messaggio di ricerca completata. Gli eventuali errori sono intercettati e mostrati in rosso come "Errore DNS".

3.4.5 list_interfaces()

La funzione mostra le interfacce di rete del sistema, eseguendo il comando `ip a` tramite `subprocess.run` con `capture_output=True` (per catturare l'output) e `text=True` (per ottenere testo leggibile anziché byte).

3.4.6 port_scan(host, ports)

La funzione riceve l'host da analizzare e una stringa con l'intervallo di porte nel formato inizio-fine (ad esempio 1-1000) ed esegue una **scansione TCP connect**. Viene inoltre creato un dizionario `results`, destinato a contenere lo stato di ciascuna porta.

Animazione di attesa

Poiché una scansione può richiedere un tempo considerevole, il programma mostra una breve animazione. La funzione interna `loading(stop_event)` scrive ripetutamente sulla stessa riga la dicitura "Scansione in corso" seguita da un carattere che varia ciclicamente tra `\`, `|` e `/`, producendo l'effetto di una ruota rotante. Utilizza `\r` (ritorno a capo senza nuova riga) per sovrascrivere la riga, `sys.stdout.flush()` per forzare la visualizzazione immediata e `time.sleep(0.1)` per regolare la velocità.

L'animazione viene eseguita in un **thread separato** ed è controllata da un `threading.Event`: finché l'evento non viene attivato il ciclo prosegue; quando viene attivato, la riga viene ripulita con degli spazi e il thread termina.

Scansione vera e propria

All'interno di un blocco `try`, per ciascuna porta dell'intervallo il programma:

1. crea un nuovo socket TCP IPv4;
2. imposta un timeout di 2 secondi;
3. chiama `s.connect_ex((host, port))`.

Il metodo `connect_ex` è analogo a `connect`, ma in caso di fallimento restituisce un codice di errore anziché sollevare un'eccezione. Se il valore restituito è 0 la connessione è riuscita: la porta è **aperta** e viene registrata nel dizionario come "Aperta"; in caso contrario viene registrata come "Chiusa". Il blocco `finally` garantisce che, in ogni circostanza, l'evento di stop venga attivato e il thread dell'animazione venga atteso con `join()`, evitando che l'animazione resti in esecuzione nel terminale.

Al termine della scansione viene stampata una tabella con lo stato delle porte, colorata secondo la convenzione indicata. La funzione restituisce inoltre il dizionario dei risultati; gli errori generali sono stampati in rosso.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

La tecnica impiegata è denominata *TCP connect scan*: per ciascuna porta il sistema tenta di completare l'handshake TCP a tre vie (SYN, SYN/ACK, ACK). Se vi riesce, significa che sulla porta è in ascolto un servizio.

3.4.7 http_test(host)

La funzione consente di inviare richieste HTTP a un host e di osservarne la risposta. Presenta un breve menu:

1. **GET** – richiede il contenuto di una risorsa (è la richiesta più comune nei browser);
2. **POST** – invia dati al server (nel caso specifico, la stringa "test");
3. **DELETE** – richiede la rimozione di una risorsa;
4. **HEAD** – analoga a GET, ma richiede soltanto le intestazioni e non il corpo;
5. **OPTIONS** – chiede al server quali metodi supporta per la risorsa.

Dopo la scelta dell'utente, il programma costruisce l'indirizzo nella forma `http://<host>` e invia la richiesta con il metodo corrispondente della libreria `requests`, con un timeout di 5 secondi per evitare attese indefinite. Per le richieste POST, DELETE, HEAD e OPTIONS stampa in verde il messaggio di successo insieme al **codice di stato HTTP** ricevuto (ad esempio 200, 404, 405); per la GET stampa la conferma di completamento. Se la scelta non è tra quelle previste viene mostrato il messaggio "Scelta non riconosciuta"; qualsiasi errore di rete viene mostrato come "Errore HTTP".

3.5 Blocco principale e menu

Il blocco `if __name__ == "__main__":` viene eseguito all'avvio del file. Richiama dapprima `print_title()` ed entra quindi nel ciclo infinito del menu, che propone sette opzioni:

Tabella 7 – Opzioni del menu principale.

Scelta	Azione	Funzione richiamata
1	Verificare la raggiungibilità di un host	`ping(host)`
2	Eseguire una scansione della rete	`scan()`
3	Verificare i record DNS di un dominio	`dns_lookup(domain)`
4	Elencare tutte le interfacce	`list_interfaces()`
5	Scansionare porte specifiche	`port_scan(host, ports)`
6	Testare HTTP con GET/POST/DELETE/HEAD/OPTIONS	`http_test(host)`
7	Uscire	Interrompe il ciclo

L'input dell'utente viene letto con `input()` e confrontato con le stringhe "1"–"7" mediante una catena `if / elif`. Per le opzioni che richiedono un bersaglio (1, 3, 5, 6) il programma chiede preventivamente l'indirizzo o il dominio dell'host. L'opzione 2 richiede la subnet all'interno della funzione stessa.

Per comodità, dopo ogni scansione delle porte l'indirizzo utilizzato viene memorizzato come attributo della funzione (`port_scan.previous_ip = host`). Alla successiva scelta dell'opzione 5 il programma verifica con `hasattr` se tale attributo esista e, in caso affermativo, chiede all'utente se intenda riutilizzare lo stesso IP, evitando di doverlo digitare nuovamente. Viene quindi richiesto l'intervallo di porte da scansionare (ad esempio 1-1000) e viene avviata `port_scan`.

3.6 Uso etico e legale

Lo strumento è destinato esclusivamente all'utilizzo su macchine proprie o per le quali si disponga di un'autorizzazione esplicita. La scansione di reti o sistemi altrui senza autorizzazione può configurare illeciti di natura civile e penale.

Zero-Day Smokey S.r.l. declina ogni responsabilità per gli usi impropri dello strumento, che ricadono interamente sull'utente finale.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

4. RACCOMANDAZIONI DI SICUREZZA

Come richiesto dalla Compagnia Theta, nella presente sezione sono formulate alcune raccomandazioni di sicurezza relative all'infrastruttura di rete e alla configurazione dei firewall.

4.1 Infrastruttura di rete

Nell'infrastruttura di rete è stata implementata la ridondanza secondo il paradigma Collapsed Core, strutturato su quattro switch core (due per sede) e basato sul protocollo Cisco HSRP (Hot Standby Router Protocol).

Interventi consigliati

Nessun rilievo.

4.2 Firewall

Criticità rilevata

È stata rilevata una criticità nella configurazione delle regole firewall, che potrebbe costituire una vulnerabilità qualora individuata da un attaccante. La DMZ, nella configurazione adottata, non può raggiungere i PC della LAN, ma può raggiungere il firewall stesso. Un attaccante che riuscisse ad accedere al firewall potrebbe disattivare tutte le regole di blocco e, potenzialmente, penetrare indisturbato nell'infrastruttura di rete.

La regola Pass ha infatti come destinazione "Any", che include anche l'indirizzo di pfSense nella DMZ (192.168.2.1), sul quale risponde l'interfaccia web di amministrazione. Dagli screenshot risulta inoltre che la password dell'account admin è ancora quella predefinita. Un attaccante che compromettesse il web server potrebbe pertanto accedere al firewall e disattivare la regola di blocco, annullando l'isolamento della DMZ.

Interventi consigliati

1. Aggiungere sulla LAN2, al di sopra della regola Pass, una regola Block con sorgente "LAN2 subnets", destinazione "This Firewall (self)" e porte TCP 80 (HTTP), 443 (HTTPS) e 22 (SSH).
2. Modificare la password dell'account admin da System → User Manager.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

5. STRUMENTO DI CATTURA DEI SOCKET DI RETE (SNIFFER)

La presente sezione illustra il funzionamento di un packet sniffer scritto in Python mediante raw socket (esclusivamente libreria standard, solo Linux), che cattura i frame dalla scheda di rete e ne decodifica gli header Ethernet, IP e di trasporto (TCP, UDP, ICMP).

5.1 Funzionamento in sintesi

Il programma opera in tre passaggi, ripetuti per ogni pacchetto:

1. il socket raw riceve tutti i frame che transitano sulla scheda di rete, così come arrivano, senza che il sistema operativo li elabori;
2. dal frame grezzo legge l'header Ethernet (primi 14 byte) e, se il pacchetto è IPv4, l'header IP (20 byte successivi);
3. in base al numero di protocollo contenuto nell'header IP determina se sopra vi sia un segmento TCP (6), un datagramma UDP (17) o un messaggio ICMP (1) e ne stampa le informazioni principali (porte oppure tipo e codice).

Il principio di fondo è che un pacchetto è costituito da livelli annidati, come buste inserite l'una nell'altra: Ethernet contiene IP, che a sua volta contiene TCP, UDP o ICMP. Il programma "apre" una busta alla volta, leggendo i byte nelle posizioni corrispondenti.

5.2 Import dei moduli

```
import socket
import struct
```

Il programma utilizza soltanto due moduli della libreria standard di Python e non richiede pertanto alcuna installazione aggiuntiva.

- `socket` fornisce l'accesso alla rete a basso livello e consente di creare il raw socket che cattura i pacchetti.
- `struct` interpreta una sequenza di byte grezzi come valori Python (numeri, stringhe di byte). Gli header di rete sono campi binari di lunghezza fissa: `struct.unpack` li suddivide nei singoli campi secondo un formato prestabilito, indicato dallo sviluppatore.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

5.3 Funzioni di supporto

```
def mac_address(dati):
    return ':'.join('%02x' % b for b in dati)


def ip_address(dati):
    return '.'.join(str(b) for b in dati)
```

Le due funzioni rendono leggibili gli indirizzi, che nel pacchetto sono rappresentati da byte grezzi.

- `mac_address` riceve i 6 byte di un indirizzo MAC e li converte nel formato canonico aa:bb:cc:dd:ee:ff. Per ogni byte `b` scrive `%02x`, ossia il valore esadecimale su due cifre, e unisce il risultato con i due punti.
- `ip_address` riceve i 4 byte di un indirizzo IP e li restituisce nel formato 192.168.1.1, ossia quattro numeri decimali separati da un punto.

Le funzioni sono collocate all'inizio del codice poiché vengono richiamate più volte: una volta definite, è sufficiente richiamarle anziché ripetere ogni volta la conversione.

5.4 Creazione del socket raw

```
s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(0x0003))
```

Questa è la riga centrale del programma: crea un socket speciale che riceve i pacchetti grezzi. I tre parametri hanno il seguente significato:

- `AF_PACKET`: famiglia di socket di Linux che opera a livello 2 (collegamento dati); vengono pertanto ricevuti i frame interi, header Ethernet compreso.
- `SOCK_RAW`: richiede i pacchetti "grezzi", senza che il sistema operativo li elabori o ne rimuova gli header; i pacchetti sono ricevuti esattamente come viaggiano sul cavo.
- `socket.ntohs(0x0003)`: il valore 0x0003 corrisponde a `ETH_P_ALL`, ossia "tutti i protocolli", in modo da catturare qualsiasi pacchetto e non soltanto IPv4; `ntohs` adegua l'ordine dei byte (network byte order) affinché il valore sia interpretato correttamente.

Subito dopo viene stampato un messaggio di avvio e inizializzato a 0 il contatore `numero`, che tiene il conteggio dei pacchetti catturati.

```
print("Sniffer avviato, premi Ctrl+C per fermarlo\n")

numero = 0
```

ZERO-DAY SMOKEY NETWORK DESIGN & ENGINEERING

5.5 Ciclo di cattura e header Ethernet

```
try:
    while True:
        pacchetto, indirizzo = s.recvfrom(65535)
        numero += 1

        # header Ethernet (primi 14 byte)
        mac_dest, mac_sorg, tipo = struct.unpack('!6s6sH', pacchetto[:14])
```

Il `while True` è un ciclo infinito: il programma continua a catturare finché non viene fermato. È contenuto in un blocco `try` perché può essere interrotto con Ctrl+C, evento gestito più avanti.

- `s.recvfrom(65535)` rimane in attesa e restituisce il pacchetto successivo; 65535 è la dimensione massima del buffer, ossia il numero massimo di byte leggibili (sufficiente per qualsiasi pacchetto).
- `numero += 1` incrementa il contatore a ogni pacchetto ricevuto.

Ha quindi inizio la decodifica dal primo livello, quello **Ethernet**:

- `pacchetto[:14]` estrae i primi 14 byte, che costituiscono l'header Ethernet.
- `struct.unpack('!6s6sH', ...)` li suddivide in tre campi secondo il formato indicato: `!` indica il network byte order, `6s` sei byte (MAC di destinazione), un ulteriore `6s` sei byte (MAC sorgente) e `H` due byte (campo tipo, che indica il protocollo incapsulato).

```
        print("Pacchetto n.", numero)
        print("  MAC sorgente:", mac_address(mac_sorg), "-> MAC destinazione:", mac_address(mac_dest))

        # 0x0800 = IPv4, gli altri tipi li salto
        if tipo != 0x0800:
            print("  Non e' un pacchetto IPv4\n")
            continue
```

Viene stampato il numero del pacchetto insieme ai due indirizzi MAC (tramite la funzione `mac_address` illustrata in precedenza). Viene quindi controllato il campo `tipo`: il valore 0x0800 identifica IPv4. Se il pacchetto non è di tipo IPv4 (ad esempio ARP o IPv6) il fatto viene segnalato e, con l'istruzione `continue`, si passa all'iterazione successiva senza tentare la decodifica.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

5.6 Decodifica dell'header IP

```
# header IP (20 byte dopo l'Ethernet)
ip_header = pacchetto[14:34]
ip = struct.unpack('!BBHHHBBH4s4s', ip_header)

lunghezza_header = (ip[0] & 15) * 4
ttl = ip[5]
protocollo = ip[6]
ip_sorg = ip_address(ip[8])
ip_dest = ip_address(ip[9])

print("  IP sorgente:", ip_sorg, "-> IP destinazione:", ip_dest, " TTL:", ttl)

inizio = 14 + lunghezza_header
```

Se il pacchetto è IPv4, si procede con la decodifica del secondo livello, quello **IP**:

- `pacchetto[14:34]` estrae i 20 byte dell'header IP, ossia quelli che seguono i 14 dell'header Ethernet.
- `struct.unpack('!BBHHHBBH4s4s', ...)` li suddivide nei campi dell'header: `B` indica un byte, `H` due byte, `4s` quattro byte. Non tutti i campi sono necessari, ma devono essere indicati tutti per poter leggere correttamente quelli di interesse.

Dai campi estratti vengono ricavate le informazioni utili:

- `lunghezza_header = (ip[0] & 15) * 4`: il primo byte contiene due informazioni; con `& 15` si mantengono i soli 4 bit meno significativi e il valore si moltiplica per 4 per ottenere la lunghezza in byte. L'operazione è necessaria perché l'header IP può avere lunghezza variabile.
- `ttl = ip[5]`: Time To Live, ossia il numero di router che il pacchetto può ancora attraversare.
- `protocollo = ip[6]`: numero che identifica il protocollo incapsulato (6 = TCP, 17 = UDP, 1 = ICMP).
- `ip_sorg` e `ip_dest`: indirizzi IP di partenza e di arrivo, convertiti in forma leggibile con `ip_address`.

Infine viene calcolato `inizio = 14 + lunghezza_header`, ossia la posizione in cui terminano gli header Ethernet e IP e inizia il livello di trasporto. Viene utilizzata la lunghezza reale dell'header IP e non un valore fisso, così che il calcolo resti corretto anche in presenza di header di lunghezza variabile.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

5.7 Decodifica di TCP, UDP e ICMP

```
    # protocollo 6 = TCP
    if protocollo == 6:
        tcp = struct.unpack('!HHLL', pacchetto[inizio:inizio + 12])
        print("  TCP  porta", tcp[0], "->", tcp[1])

    # protocollo 17 = UDP
    elif protocollo == 17:
        udp = struct.unpack('!HHHH', pacchetto[inizio:inizio + 8])
        print("  UDP  porta", udp[0], "->", udp[1])

    # protocollo 1 = ICMP
    elif protocollo == 1:
        icmp = struct.unpack('!BBH', pacchetto[inizio:inizio + 4])
        print("  ICMP tipo", icmp[0], "codice", icmp[1])

    else:
        print("  Altro protocollo:", protocollo)

    print()
```

Questo è il terzo livello, quello di **trasporto**, incapsulato nel pacchetto IP. Viene impiegato il costrutto `if / elif / else` perché un pacchetto contiene uno solo di tali protocolli, individuato in base al numero letto in precedenza. In tutti i casi la lettura parte da `inizio`, la posizione calcolata nel paragrafo precedente.

- **TCP (protocollo 6)**: legge i primi 12 byte dell'header con il formato `!HHLL` (due campi da 2 byte e due da 4). I primi due corrispondono alla porta sorgente (`tcp[0]`) e alla porta di destinazione (`tcp[1]`).
- **UDP (protocollo 17)**: legge 8 byte con `!HHHH`; anche in questo caso i primi due campi sono le porte. UDP è più semplice di TCP e il suo header è più corto.
- **ICMP (protocollo 1)**: protocollo utilizzato dal ping. Legge 4 byte con `!BBH` ed estrae tipo e codice, i due campi che identificano il messaggio (ad esempio richiesta o risposta echo).
- **else**: se il protocollo è diverso, stampa soltanto il numero senza decodificarlo.

Alla fine `print()` stampa una riga vuota per separare un pacchetto dal successivo e rendere più leggibile l'output.

5.8 Esecuzione e note d'uso

Uscita dal ciclo

```
except KeyboardInterrupt:
    print("\nSniffer fermato. Pacchetti catturati:", numero)
    s.close()
```

Quando l'utente preme **Ctrl+C**, Python solleva l'eccezione `KeyboardInterrupt`. Il blocco `except` la intercetta, stampa il numero totale di pacchetti catturati e chiude il socket con `s.close()`, liberando la risorsa.

Esecuzione

```
sudo python3 sniffer.py
```

Per visualizzare i pacchetti ICMP è sufficiente aprire un secondo terminale ed eseguire un ping, ad esempio `ping 8.8.8.8`. Gli screenshot acquisiti durante il test sono riportati di seguito.

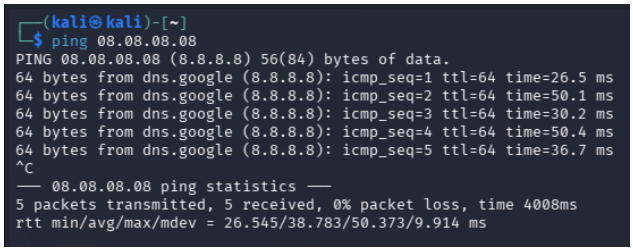


Figura 8 – *Esecuzione del comando ping verso 8.8.8.8 da un secondo terminale.*

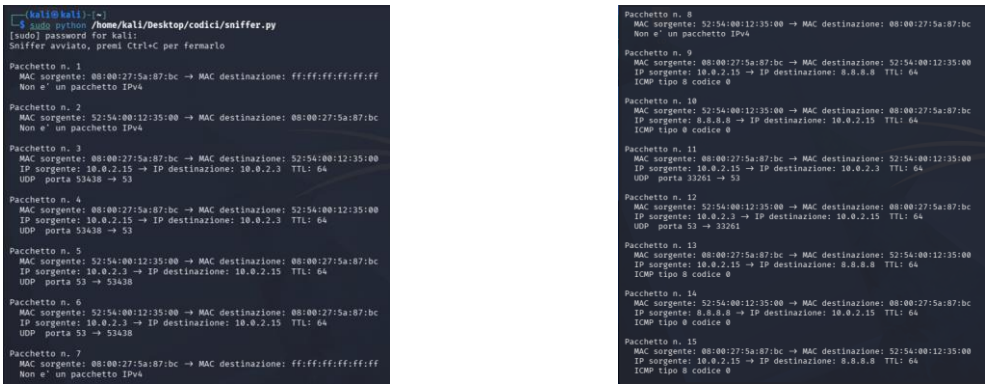


Figura 9 – *Output dello sniffer durante il test: cattura di frame non IPv4, datagrammi UDP e messaggi ICMP.*

Motivo dell'utilizzo di sudo

I raw socket consentono di leggere tutto il traffico che transita dalla scheda di rete e costituiscono pertanto un'operazione privilegiata: Linux li concede soltanto all'utente **root**. Per tale ragione il programma deve essere avviato con `sudo`; in caso contrario viene restituito un errore di permessi.

Nota sull'uso lecito

Lo sniffer deve essere utilizzato esclusivamente sulla propria rete, in un ambiente di laboratorio o in presenza di un'autorizzazione esplicita. L'intercettazione del traffico di reti o dispositivi altrui senza autorizzazione non è consentita.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

6. PREVENTIVO DI SPESA

Il preventivo n. 00001 comprende le sole forniture e prestazioni tecniche relative all'infrastruttura descritta nei capitoli precedenti. Gli importi sono in euro; l'IVA è calcolata al 22%.

Tabella 9 – Preventivo complessivo n. 00001.

Descrizione	Prezzo	Quantità	Imponibile
Sede 1 – Edificio a sei piani (120 postazioni)			
Switch di accesso 24 porte con DHCP (uno per piano)	459,50 €	6	2757,00 €
Web server (DVWA) in DMZ	3684,00 €	1	3684,00 €
Firewall perimetrale	981,63 €	1	981,63 €
NAS	952,00 €	1	952,00 €
IPS dedicato (tra firewall e DVWA)	780,00 €	1	780,00 €
Router edge con funzione IPS integrata	297,00 €	2	594,00 €
Switch Multilayer 24 porte PoE con 4 porte SFP	1305,00 €	2	2610,00 €
Cablaggio strutturato	8,00 €	5000 m	40.000,00 €
Sede 2 – Edificio a tre piani (90 postazioni)			
Switch di accesso 48 porte con uplink SFP+ (uno per piano)	535,00 €	3	1605,00 €
Switch Multilayer 24 porte PoE con 4 porte SFP	1305,00 €	2	2610,00 €
Cablaggio strutturato	8,00 €	2500 m	20.000,00 €
Collegamento in fibra ottica tra le sedi (circa 250 m)			
Pratica e progettazione	a corpo	1	1800,00 €
Fibra ottica multimodale	5,00 €	250 m	1250,00 €
Modulo ottico SFP multimodale 850 nm, LC duplex	35,00 €	4	140,00 €
Collaudo e certificazione del collegamento	a corpo	1	1500,00 €
Manodopera			
Installazione, configurazione e messa in servizio	a corpo	1	12.000,00 €
Totale imponibile			**93.263,63 €**
IVA 22%			**20.518,00 €**
Totale preventivo			**113.781,63 €**

- **Variazioni di spesa**: la spesa può variare qualora emergano problematiche non prevedibili (ad esempio difformità o inaccessibilità di locali e cavidotti, prescrizioni degli enti, ritardi non imputabili all'Appaltatore); i maggiori oneri sono a carico del Cliente, previa approvazione scritta.
- **Esclusioni**: opere civili e murarie, oneri comunali per i permessi e spese non indicate, a carico del Cliente.
- **Pagamento e validità**: pagamento entro 30 giorni dall'accettazione (acconto 30%, saldo entro 15 giorni); validità 30 giorni dall'emissione; tempi di consegna definiti alla conferma dell'incarico.

ZERO-DAY SMOKEY NETWORK DESIGN & ENGINEERING

7. ENIGMA

La presente sezione riporta la risoluzione dell'enigma proposto dal Cliente, consistente nell'analisi della cartella di file ricevuta.

7.1 Analisi della cartella file della Compagnia Theta

L'analisi ha riguardato una sfida di steganografia a catena, nascosta in una copia modificata del template "Multiverse" di HTML5 UP: dodici immagini JPEG con payload cifrati, ciascuna delle quali rende disponibile la password per l'immagine successiva. La flag finale, **DRAGON34**, è un'immagine ricomposta a partire da frammenti codificati in base64, suddivisi con il comando `split` e distribuiti tra più fotografie.

7.1.1 Analisi degli indizi in chiaro

HTML

Tra le descrizioni fittizie delle fotografie, due articoli (09 "Prima" e 10 "Seconda") contenevano indovinelli in rima anziché testo di riempimento:

1. *"Nascosto nella notte e avvolto nel silenzio… un protagonista ardente… Chi sono io?"* → tema del **mistero** (il giallo, il detective);
2. *"…porto la chiarezza da un labirinto… arrivo al termine di un lungo viaggio di mente"* → tema del **risolto** (la soluzione).

Le due parole chiave sono dunque: mistero + risolto.

JavaScript

Nel commento di testa del file è comparsa una riga aggiunta: `? mistero risolto ?`, a conferma della prima password.

CSS

Nel file `main.css` è stato individuato un commento nascosto:

```
/* Sei sicuro di aver finito? Pensa a meno elicotteri e meno 64 */
```

Il commento costituisce un indizio per gli stadi successivi: "elicottero" rimanda alla foto 09, "64" alla codifica base64.

Si è ipotizzato che le immagini JPEG contenessero dati cifrati protetti da password; per l'estrazione è stato pertanto utilizzato **steghide**.

ZERO-DAY SMOKEY NETWORK DESIGN & ENGINEERING

7.1.2 La catena di sblocco

Ogni immagine contiene la password della successiva: si tratta di una catena e non di immagini indipendenti. L'estrazione avviene con il seguente comando:

```
steghide extract -sf <img> -p <password>
```

Tabella 8 – Catena di sblocco delle immagini.

Ordine	Immagine	Password	Contenuto estratto
1	01.jpg	mistero risolto	pass.txt – codice Brainfuck
2	02.jpg	D5f89fgg	help.txt – codice COW
3	03.jpg	D5f89fgg	PolyjuicePotion.png – falso PNG, file di credenziali
4	04, 05, 06, 07, 11.jpg	Aswap78122	Frammenti base64 flag-aa … flag-ad, flag-ah
5	09, 10.jpg	Ada81QlAsKL	Frammenti base64 flag-ae, flag-af

Il file di credenziali del passo 3 conteneva le chiavi per le fasi successive.

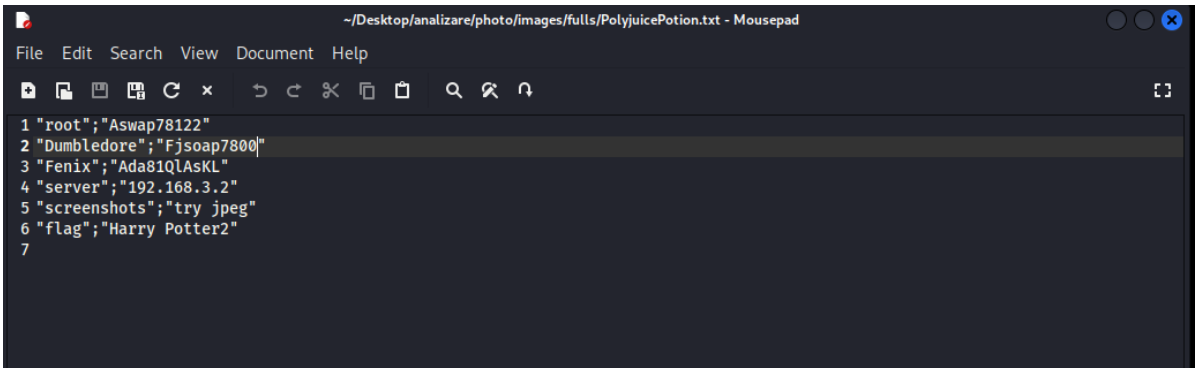


Figura 10 – *Contenuto del file di credenziali estratto dall'immagine 03.jpg.*

Un dettaglio rilevante: i frammenti sono denominati flag-aa, flag-ab, …, flag-ah, ossia esattamente l'output del comando Unix `split`. Un singolo file è stato suddiviso in otto parti e ciascuna parte è stata nascosta in un'immagine diversa.

7.1.3 Decodifica dei linguaggi non convenzionali

Due stadi utilizzavano linguaggi di programmazione non convenzionali.

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

Brainfuck (01.jpg)

Il contenuto era costituito unicamente da simboli. Eseguito con un interprete Brainfuck, produce la stringa **D5f89fgg**, ovvero la password dello stadio 2.

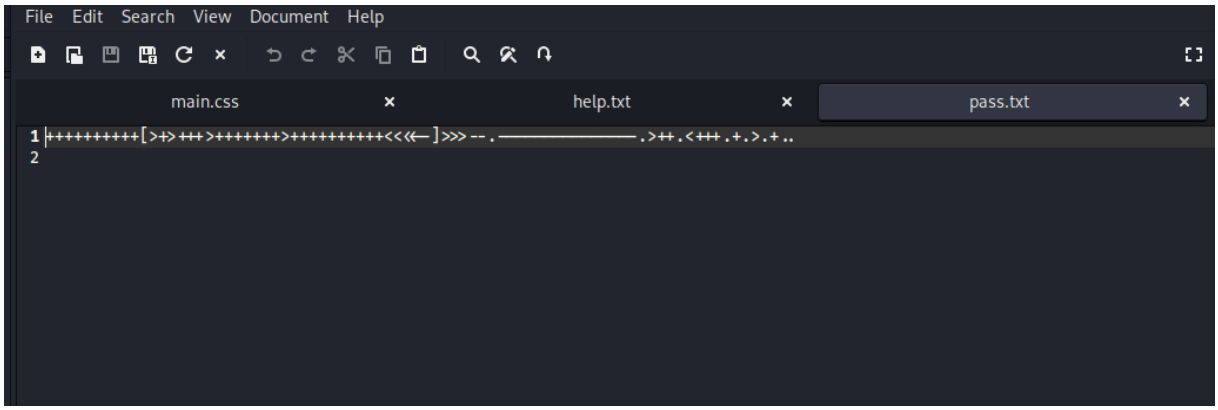


Figura 11 – *Codice Brainfuck contenuto nel file pass.txt.*

COW (02.jpg)

Il contenuto era costituito dai token moo, mOo, MOO, OOO..., tipici della sintassi del linguaggio COW. Eseguito con un interprete COW, produce la parola **base64**, indizio che indicava come trattare i frammenti dello stadio finale.

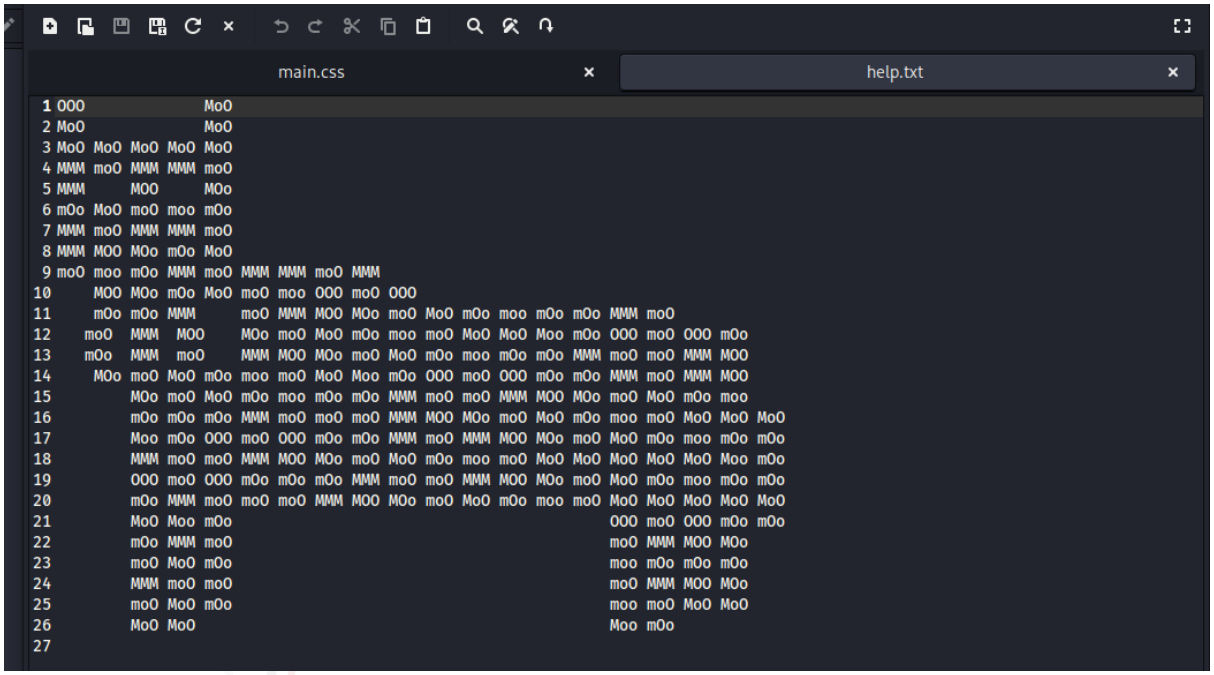


Figura 12 – *Codice COW contenuto nel file help.txt.*

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING

7.1.4 Ricomposizione e flag finale

Utilizzando le password sopra indicate, “Aswap78122” e “Ada81QlAsKL”, sulle immagini .jpg (cfr. tabella precedente), sono state estratte le flag come illustrato nelle figure seguenti.



Figura 13 – *Frammenti estratti dalle immagini, con estensione .png.*

Poiché i file .png non erano immagini valide e non venivano letti dal sistema, ne è stata modificata l’estensione in .txt.

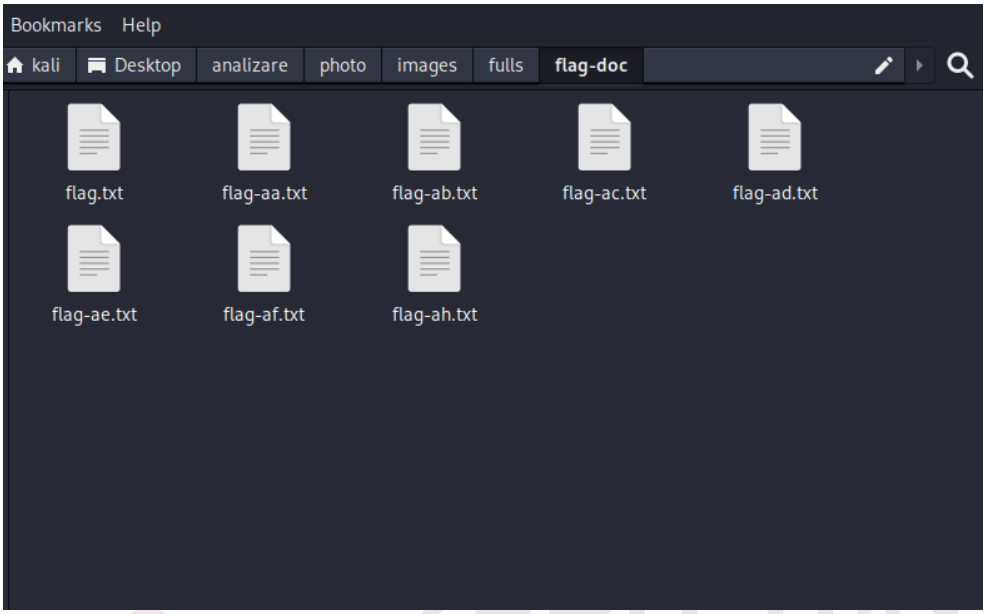


Figura 14 – *Frammenti dopo la modifica dell’estensione in .txt.*

ZERO-DAY SMOKEY NETWORK DESIGN & ENGINEERING

Aprendo i file si è constatato che il contenuto era codificato in base64; i frammenti sono stati pertanto copiati e incollati in sequenza in un unico file denominato “flag.txt”.

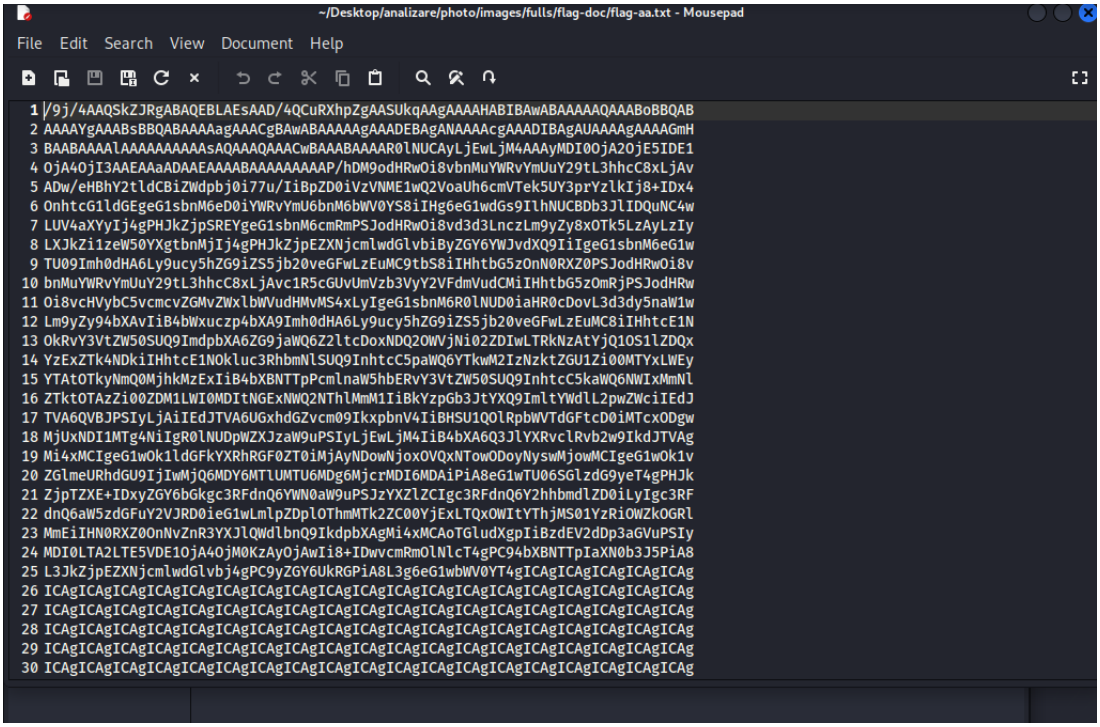


Figura 15 – *Contenuto dei frammenti codificati in base64.*

I frammenti sono stati concatenati in ordine alfabetico (flag-aa → flag-ah) e sottoposti a decodifica base64. Il risultato è un’immagine PNG raffigurante una sala server, recante la scritta **REAL FLAG: DRAGON34**, che rappresenta la soluzione della cartella “misteriosa” inviata dalla Compagnia Theta.



Figura 16 – *Immagine ricomposta contenente la flag finale.*

SMOKEY
NETWORK DESIGN & ENGINEERING

7.1.5 Recupero della password dell'ultima immagine

Per eseguire l'attacco a dizionario (brute force) su Kali Linux è stato utilizzato **stegseek**, il cracker di steghide più rapido, che opera con una wordlist e non richiede l'uso di Python. L'installazione avviene dal terminale con i seguenti comandi:

```
sudo apt update
sudo apt install stegseek
```

Dopo aver creato una wordlist contenente tutte le combinazioni possibili con la parola DRAGON34 e con le parole o password individuate nel corso della risoluzione dell'enigma, è stato eseguito il comando:

```
stegseek images/fulls/12.jpg wordlist.txt
```

La password individuata è **base DRAGON34**. Inserita la password, si apre l'immagine riportata di seguito; selezionando la voce "grazie speciale" si viene reindirizzati a un video su YouTube, corrispondente alla sigla iniziale della serie animata Futurama (https://www.youtube.com/watch?v=MWiWEJQJYag).



Figura 17 – *Immagine estratta da 12.jpg con il collegamento al video.*

ZERO-DAY SMOKEY
NETWORK DESIGN & ENGINEERING